

系统开发工具基础实验报告

第 2 周

姓名：曾奕豪

课程：系统开发工具基础

2026 年 9 月 1 日

仓库：

<https://github.com/hzydy00/Base-of-system-development-tools>

1 课上内容与结果

学习主题

命令行环境；开发环境与工具；Debugging and Profiling

实验环境

本实验基于 Windows 11 操作系统，实验涉及的 Shell 操作均在 Git Bash 中完成；版本控制使用 Git；LaTeX 文档采用 XeLaTeX 编译，并通过 ctex 宏包中文排版。

题目一

第 5 题 控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 建议用时：12—15 分钟

将下方脚本保存为 q05/worker.sh，再完成任务控制。

给定内容

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
  echo "$n"
  n=$((n+1))
  sleep 1
done
```

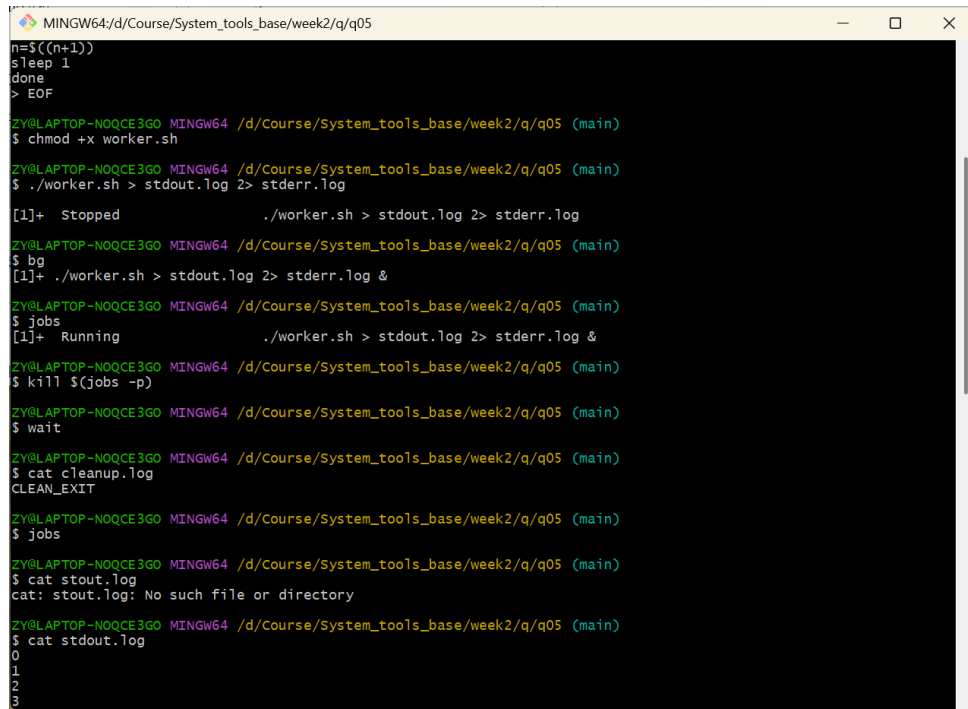
题目要求

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9。

题目一 题目要求

```
1 # TODO: 题目一代码汇总
2 cat > worker.sh <<'EOF'
3 > #!/usr/bin/env bash
4 trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
5 n=0
6 while true; do
7 echo "$n"
8 n=$((n+1))
9 sleep 1
10 done
11 > EOF
12
13 chmod +x worker.sh
14
15 ./worker.sh > stdout.log 2> stderr.log
16 [1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
17
18 bg #后台运行
19 [1]+  ./worker.sh > stdout.log 2> stderr.log &
20
21 jobs #确认状态
22 [1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
23
24 kill (jobs -p) #杀死进程
25
26
27 cat cleanup.log #获取日志, 返回 CLEAN_EXIT
28
29 jobs
30
31 cat stdout.log
```

部分实验过程截图



```
MINGW64/d/Course/System_tools_base/week2/q/q05
n=$((n+1))
sleep 1
done
> EOF

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ chmod +x worker.sh

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ ./worker.sh > stdout.log 2> stderr.log

[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ kill $(jobs -p)

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ wait

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ cat cleanup.log
CLEAN_EXIT

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ jobs

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ cat stout.log
cat: stout.log: No such file or directory

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q05 (main)
$ cat stdout.log
0
1
2
3
```

题目一实验过程截图

实验结果

1. 输出重定向效果：脚本运行期间，‘stdout.log’ 会每秒追加一行递增的整数（0、1、2...）；脚本无错误输出，因此 ‘stderr.log’ 保持为空。
2. 任务状态变化：Ctrl+Z 后任务进入 ‘Stopped’（暂停）状态；执行 ‘bg’ 后转为 ‘Running’（后台运行）状态，‘jobs’ 命令可看到作业编号与对应状态。
3. 信号与优雅退出：发送 SIGTERM 后，进程被 trap 捕获信号，执行日志写入逻辑后正常退出；cleanup.log 文件中会出现一行 CLEAN_EXIT。

题目二

第 6 题 语义重构与本地开发反馈

主题：开发环境与工具 建议用时：12—15 分钟

在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成一次语义重构。

给定内容

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

题目要求

第 4 页

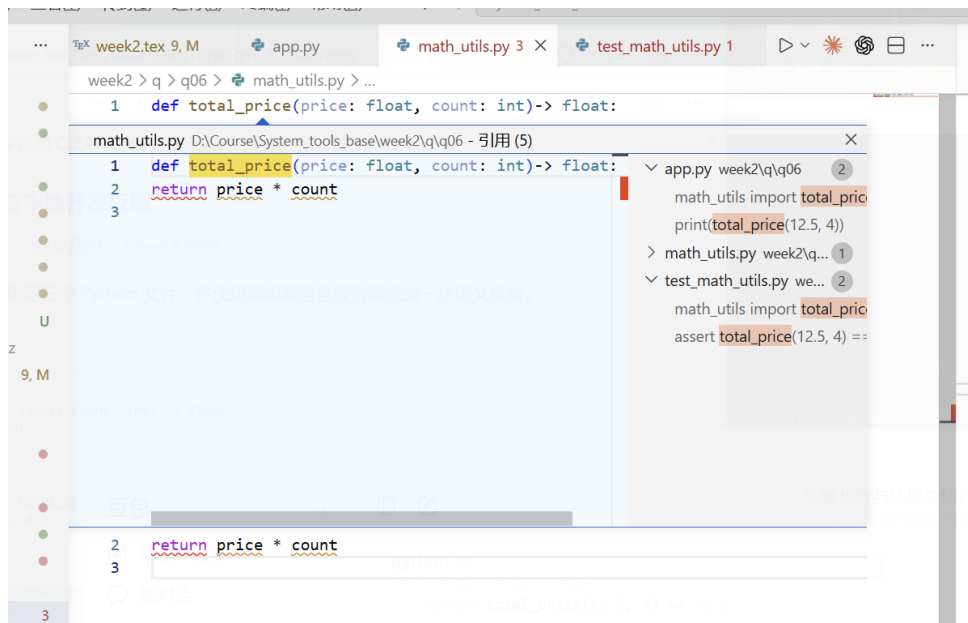
系统开发工具基础 · 每周课上实验检查题

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 total_price 改为 calculate_total，保证定义和所有代码引用同步改变。
- 在 app.py 中临时加入未使用的 import os，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

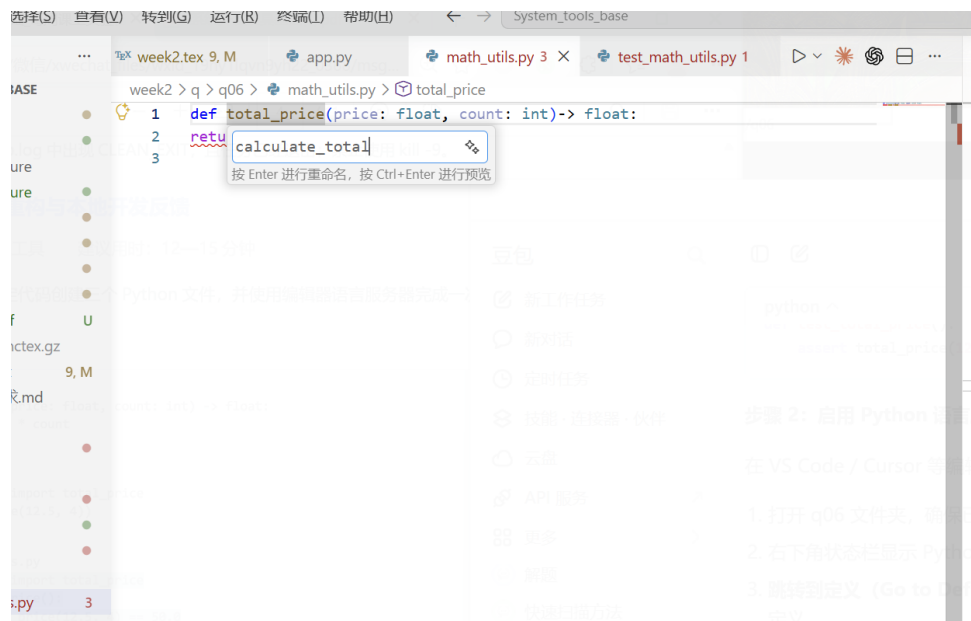
题目二题目要求

```
1 # TODO: 题目二代码汇总
2
3 #导入 math_utils.p, app.py, test_math_utils.py 文件
4
5 #按 F2 重命名符号, shift+F12 查找, ctrl+点击跳转
6
7 #import os 后, ruff 检查与一键修复
8 python -m ruff check .
9
10 python -m ruff check --fix .
11
12 #pytest 测试通过
13 python -m pytest test_math_utils.py -v
```

部分实验过程截图



题目二实验过程截图



题目二实验过程截图

```

MINGW64/d/Course/System_tools_base/week2/q/q06
3 | from math_utils import calculate_total
4 |
5 | print(calculate_total(12.5, 4))

F401 [*] 'os' imported but unused
--> app.py:1:8
1 | import os
  |     ^^
2 | from math_utils import calculate_total
3 | print(calculate_total(12.5, 4))

help: Remove unused import: 'os'
- import os
1 | from math_utils import calculate_total

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1 | from math_utils import calculate_total
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 | def test_calculate_total():
3 |     assert calculate_total(12.5, 4) == 50.0

help: Organize imports
1 | from math_utils import calculate_total
2 |
3 |
4 | def test_calculate_total():

Found 3 errors.
[*] 3 fixable with the '--fix' option.

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q06 (main)
$ python -m ruff check --fix .
Found 3 errors (3 fixed, 0 remaining).

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q06 (main)
$

```

题目二实验过程截图

```

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q06 (main)
$ python -m ruff check --fix .
Found 3 errors (3 fixed, 0 remaining).

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q06 (main)
$ python -m pytest test_math_utils.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- D:\python\python.exe
cachedir: .pytest_cache
rootdir: D:\Course\System_tools_base\week2\q\q06
plugins: anyio-4.13.0
collected 1 item

test_math_utils.py::test_calculate_total PASSED [100%]

===== 1 passed in 0.02s =====

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q06 (main)
$

```

题目二实验过程截图

实验结果

1. 语义重构（重命名符号）后，三个文件的定义和引用全部同步更新，程序功能不受影响，输出仍为 '50.0'。
2. ruff 成功识别未使用的 'import os'（F401 规则），并通过 '-fix' 自动删除。
3. 最终 ruff 零报错、pytest 全部通过，满足题目所有要求。

题目三

第 7 题 用调试器定位归并排序缺陷

主题: Debugging 建议用时: 12—15 分钟

将下列存在缺陷的归并排序代码保存为 q07/merge_sort.py, 再使用调试器定位并修复。

给定内容

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

题目要求

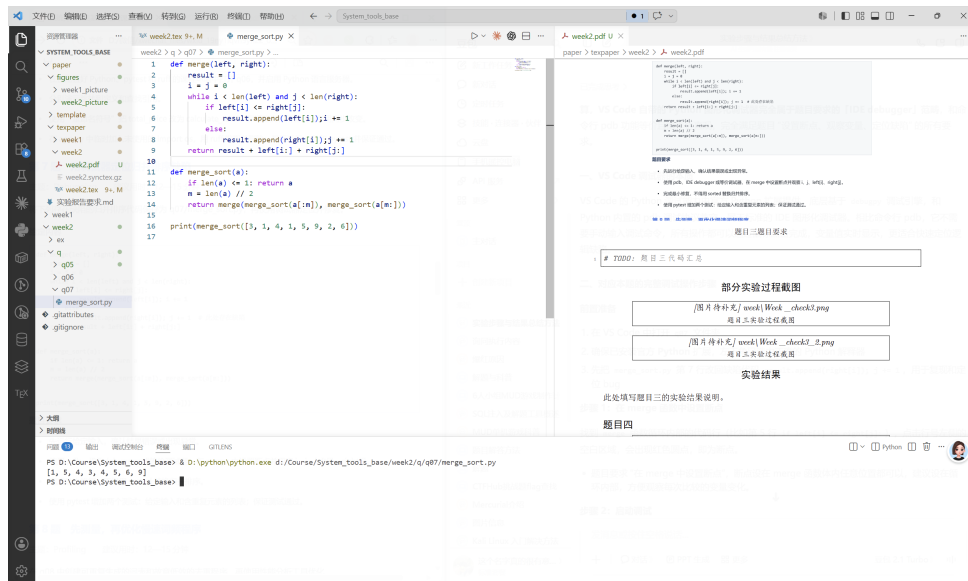
- 先运行给定输入, 确认结果错误或出现异常。
- 使用 pdb、IDE debugger 或等价调试器, 在 merge 中设置断点并观察 i、j、left[i]、right[j]。
- 完成最小修复, 不得用 sorted 替换归并排序。
- 使用 pytest 增加两个测试: 给定输入和含重复元素的列表; 保证测试通过。

第 8 题 生成量 西岸少慢速归并排序

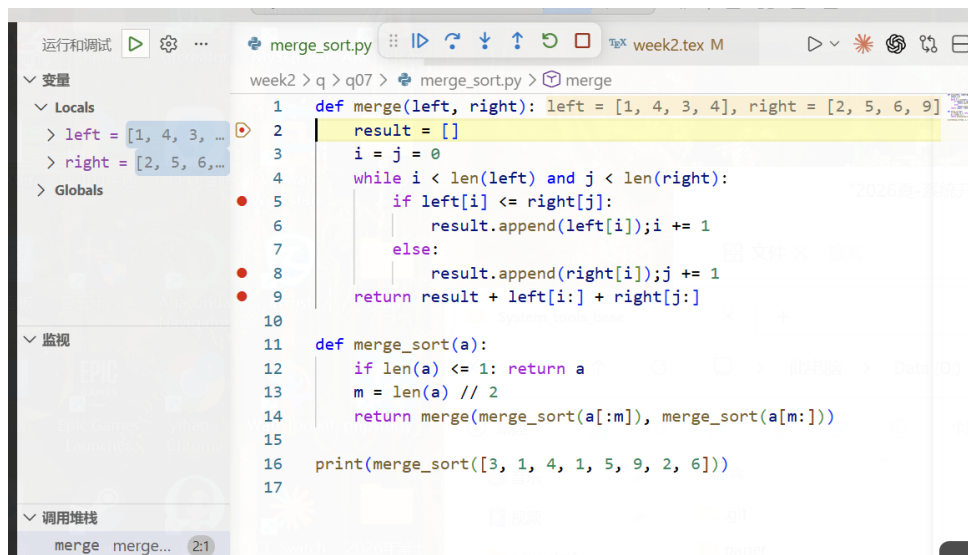
题目三题目要求

```
1 # TODO: 题目三代码汇总
2 #将原代码第8行i改为j
3 # 编写 test_merge_sort.py 文件, 使用 pytest 测试 merge_sort 函数
4 from merge_sort import merge_sort
5
6 def test_given_input():
7     """测试用例1: 题目给定的输入数组"""
8     arr = [3, 1, 4, 1, 5, 9, 2, 6]
9     # 断言: 排序后的结果等于预期的升序数组
10    assert merge_sort(arr) == [1, 1, 2, 3, 4, 5, 6, 9]
11
12 def test_duplicate_elements():
13     """测试用例2: 包含大量重复元素的数组"""
14     # 构造重复元素密集的测试用例, 验证算法稳定性和边界处理
15     arr = [2, 2, 1, 3, 2, 5, 4, 2]
16    assert merge_sort(arr) == [1, 2, 2, 2, 2, 3, 4, 5]
```

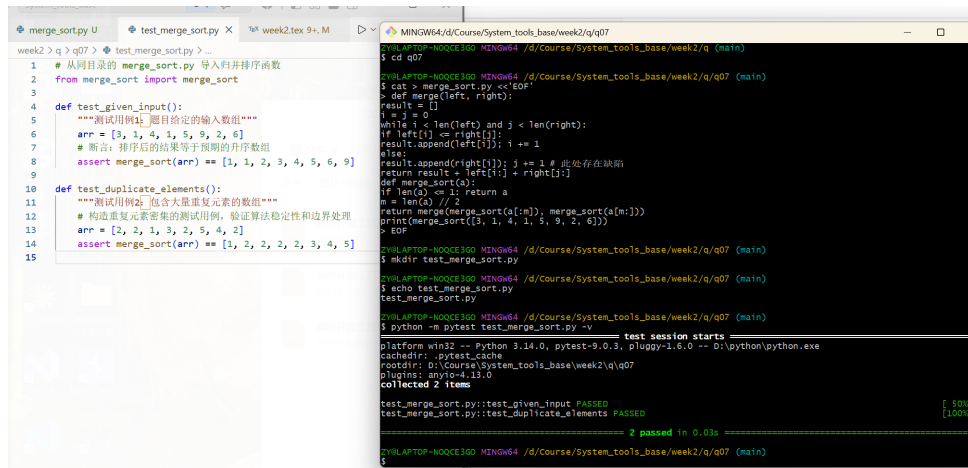
部分实验过程截图



题目三实验过程截图：确认输出结果有误



题目三实验过程截图：设置断点，发现错误并修改



The screenshot displays a VS Code editor with two files open: `merge_sort.py` and `test_merge_sort.py`. The `merge_sort.py` file contains a recursive merge sort function. The `test_merge_sort.py` file includes two pytest test cases: `test_given_input` which tests a specific input array, and `test_duplicate_elements` which tests the algorithm's stability with an array containing duplicate elements. To the right, a terminal window shows the execution of `python -m pytest test_merge_sort.py`, which successfully passes both tests, indicating the merge sort implementation is correct.

题目三实验过程截图：用 pytest 测试，确认修改后输出正确

实验结果

1. 调试定位：通过 VS Code IDE 调试器在 ‘merge’ 函数内设置断点、单步执行并观察 ‘i’、‘j’、‘left[i]’、‘right[j]’ 变量，精准定位根因：右数组元素读取与指针不匹配，属于双指针索引笔误。
2. 修复效果：执行最小修复，仅将 ‘right[i]’ 修正为 ‘right[j]’，未改动算法整体逻辑、未使用 ‘sorted’ 替换；修复后运行给定输入，输出正确升序结果 ‘[1, 1, 2, 3, 4, 5, 6, 9]’。
3. 测试验证：编写 2 个 pytest 测试用例（题目给定输入、含重复元素的数组），运行后全部通过，验证了修复后算法的基础排序功能与重复元素场景下的稳定性，完全满足题目要求。

题目四

第 8 题 先测量，再优化慢速词频程序

主题: Profiling 建议用时: 12—15 分钟

在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化。

给定内容

```
# generate_words.py
import random
```

第 5 页

系统开发工具基础 · 每周课上实验检查题

```
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

题目要求

- 生成 words.txt，使用 time 运行原程序 2 次并记录耗时。
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置。
- 在不改变最终 count 的前提下优化程序；不得写死 1000。
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

题目四题目要求

```
1 # TODO: 题目四代码汇总
2
3 #计算优化前的程序耗时
4 time python wordfreq.py
5
6 #集合去重
7 # wordfreq_optimized.py
8 words = open("words.txt", encoding="utf-8").read().split()
9 unique = set(words)
10 print("count=", len(unique))
11
12 #计算优化后的程序耗时
13 time python wordfreq_optimized.py
```

部分实验过程截图

```

MINGW64:/d/Course/System_tools_base/week2/q/q08
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ python generate_words.py

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.328s
user    0m0.030s
sys     0m0.000s

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.313s
user    0m0.000s
sys     0m0.046s

```

题目四实验过程截图: 优化前中位耗时 = $(0.313 + 0.328) \div 2 = 0.3205$ 秒

```

MINGW64:/d/Course/System_tools_base/week2/q/q08
user    0m0.030s
sys     0m0.000s

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.313s
user    0m0.000s
sys     0m0.046s

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.098 seconds

Ordered by: cumulative time
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.098    0.098 {built-in method builtins.exec}
1      0.097    0.097    0.098    0.098 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {built-in method _io.open}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}

```

题目四实验过程截图: 核心耗时集中在主程序执行
(wordfreq.py:1(<module>))

题目四实验过程截图: 优化前中位耗时 = $(0.221 + 0.220) \div 2 = 0.2205$ 秒,
加速比 = $0.3205 \div 0.2205 = 1.45$

课上阶段 commit 记录

2 课后练习与结果

实例 1（开发环境与工具）

实验题目：浏览一份 IDE 扩展列表，安装一个看起来对你有用的扩展。

1 # TODO：实例1无代码汇总

运行结果：如下图，安装成功



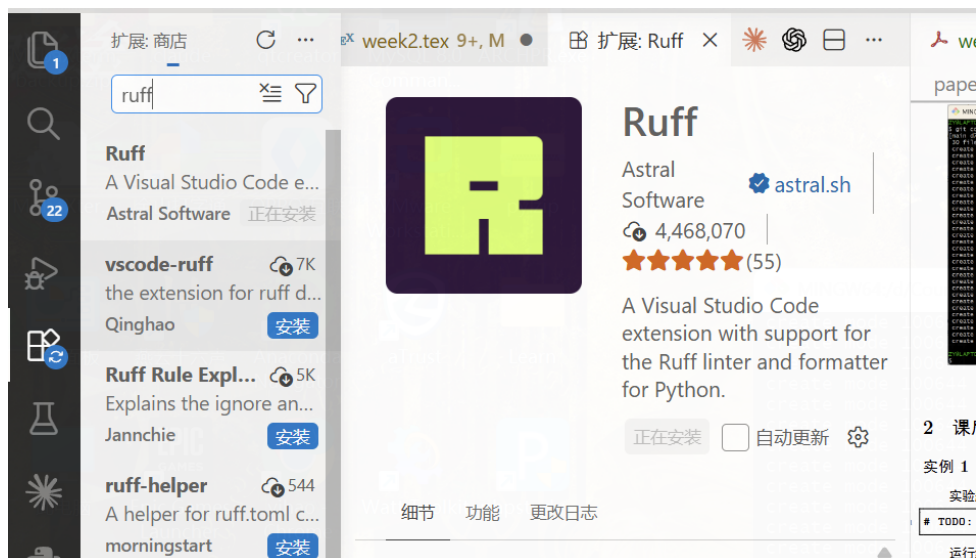
实例一运行结果

实例 2（开发环境与工具）

实验题目：为你正在做的一个项目配置 IDE 扩展和语言服务器。

1 # TODO：实例2无代码汇总

运行结果：如下图，配置成功（已配置语言服务器增强 pylance，所以下载 ruff）



实例二运行结果

实例 3（命令行环境）

实验题目：写出一个 `ls` 命令，让它按下面的方式列出文件：

- 包含所有文件，包括隐藏文件
- 文件大小以易读格式显示（例如 454M，而不是 454279954）
- 按最近修改时间排序
- 输出带颜色

```
1 # TODO: 实例3代码汇总
2 ls -laht --color=auto
```

运行结果：

```
MINGW64:/d/Course/System_tools_base/week2/q/q08
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ ls -laht --color=auto
total 147K
-rw-r--r-- 1 ZY 197121 144K Aug 31 15:25 words.txt
drwxr-xr-x 1 ZY 197121 0 Aug 31 15:25 ./
-rw-r--r-- 1 ZY 197121 227 Aug 31 15:23 wordfreq_optimized.py
-rw-r--r-- 1 ZY 197121 195 Aug 31 15:22 wordfreq.py
-rw-r--r-- 1 ZY 197121 221 Aug 31 15:22 generate_words.py
drwxr-xr-x 1 ZY 197121 0 Aug 31 15:19 ../
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week2/q/q08 (main)
$ |
```

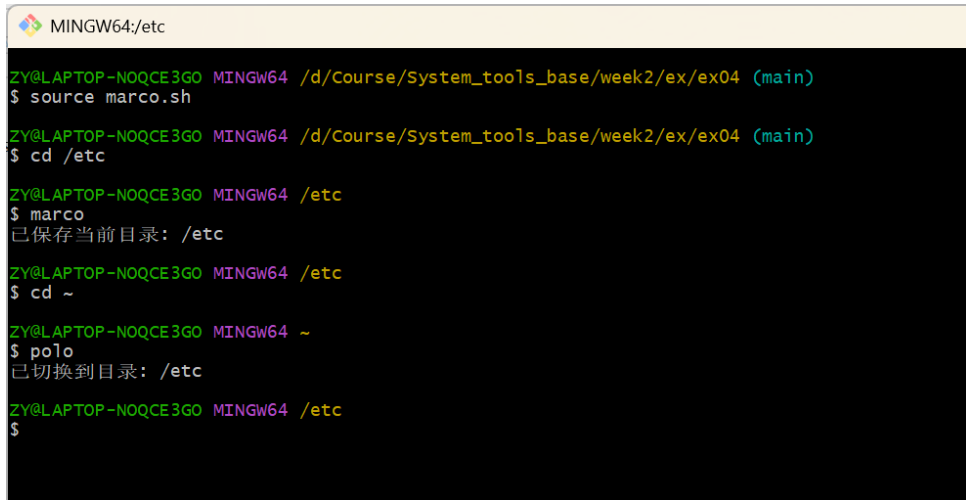
实例三运行结果

实例 4(命令行环境)

实验题目：写两个 `bash` 函数 `marco` 和 `polo`，行为如下：每次执行 `marco` 时，都要以某种方式保存当前工作目录；之后无论你切到哪个目录，只要执行 `polo`，它都应该把你 `cd` 回执行 `marco` 时所在的目录。为了方便调试，你可以把代码写进 `marco.sh`，然后通过执行 `source marco.sh` 把这些定义重新加载到当前 `shell`。

```
1 # TODO: 实例4代码汇总
2 # 将函数加载到当前 Shell
3 source marco.sh
4
5 # 完整测试流程
6 cd /etc
7 marco      # 保存 /etc 目录
8 cd ~
9 polo      # 自动切回 /etc 目录
```

运行结果：成功切回 `/etc` 目录



```

MINGW64:/etc

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week2/ex/ex04 (main)
$ source marco.sh

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week2/ex/ex04 (main)
$ cd /etc

ZY@LAPTOP-NOQCE3G0 MINGW64 /etc
$ marco
已保存当前目录: /etc

ZY@LAPTOP-NOQCE3G0 MINGW64 /etc
$ cd ~

ZY@LAPTOP-NOQCE3G0 MINGW64 ~
$ polo
已切换到目录: /etc

ZY@LAPTOP-NOQCE3G0 MINGW64 /etc
$

```

实例四运行结果

实例 5（命令行环境）

实验题目：假设你有一个很少失败的命令。为了调试它，你需要把它的输出保存下来，但等到一次失败运行可能会很耗时。写一个 bash 脚本，不断运行下面这个脚本（此脚本参考原题目）直到它失败为止，并把标准输出和标准错误分别保存到文件里，最后把结果打印出来。顺便报告它运行了多少次才失败。

```

1 # TODO: 实例5代码汇总
2 #!/usr/bin/env bash
3
4 # 配置：日志文件路径与待测脚本路径
5 STDOUT_LOG="fail_stdout.log"
6 STDERR_LOG="fail_stderr.log"
7 TARGET_SCRIPT="./faulty_cmd.sh"
8
9 # 运行次数计数器
10 run_count=0
11
12 # 无限循环执行命令，直到失败
13 while true; do
14     run_count=$((run_count + 1))
15     # 执行待测脚本，分别重定向标准输出和标准错误
16     "$TARGET_SCRIPT" > "$STDOUT_LOG" 2> "$STDERR_LOG"
17     # 退出码非0则表示失败，终止循环
18     if [[ $? -ne 0 ]]; then
19         break
20     fi

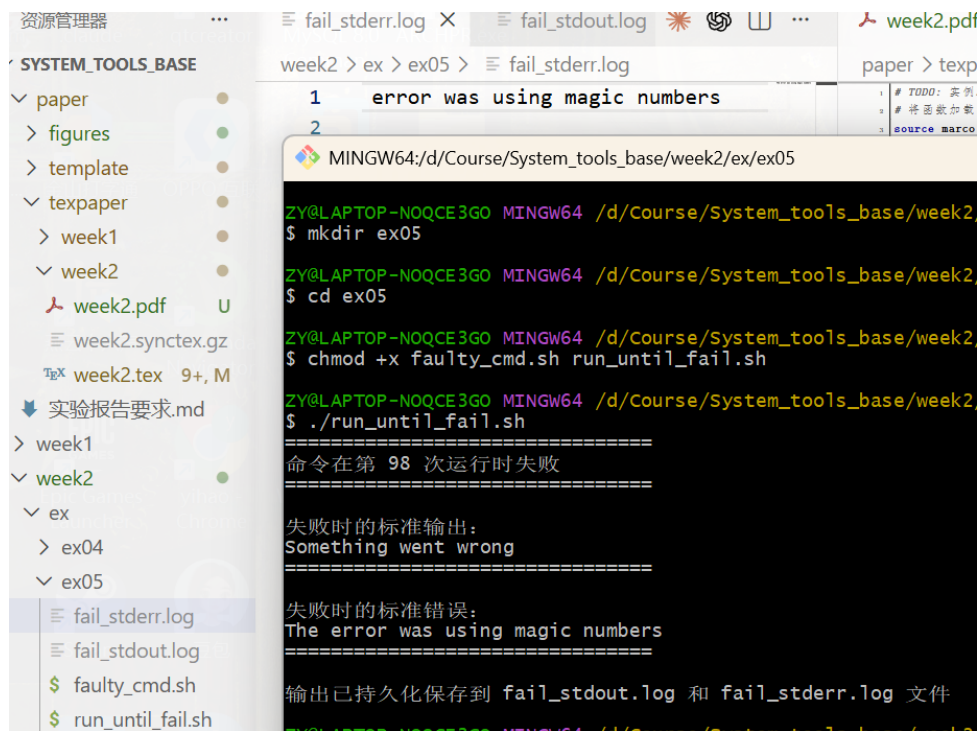
```

```

21 done
22
23 # 打印最终结果
24 echo "===== "
25 echo "命令在第 $run_count 次运行时失败"
26 echo "===== "
27 echo -e "\n失败时的标准输出: "
28 cat "$STDOUT_LOG"
29 echo "===== "
30 echo -e "\n失败时的标准错误: "
31 cat "$STDERR_LOG"
32 echo "===== "
33 echo -e "\n输出已持久化保存到 $STDOUT_LOG 和 $STDERR_LOG 文件"
34
35 # 添加执行权限
36 chmod +x faulty_cmd.sh run_until_fail.sh
37
38 # 运行测试脚本
39 ./run_until_fail.sh

```

运行结果：如下图



```

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week2
$ mkdir ex05

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week2
$ cd ex05

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week2
$ chmod +x faulty_cmd.sh run_until_fail.sh

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week2
$ ./run_until_fail.sh

=====
命令在第 98 次运行时失败
=====

失败时的标准输出:
Something went wrong
=====

失败时的标准错误:
The error was using magic numbers
=====

输出已持久化保存到 fail_stdout.log 和 fail_stderr.log 文件

```

实例五运行结果

实例 6(命令行环境)

实验题目：在终端里启动一个 `sleep 10000` 任务，用 `Ctrl-Z` 把它挂起，再用 `bg` 让它继续在后台运行。然后使用 `pgrep` 找到它的 `pid`，再用 `pkill` 把它杀掉，整个过程都不要手动输入这个 `pid`。（提示：试试 `-af` 参数）

```
1 # TODO: 实例6代码汇总
2 # 1. 启动前台 sleep 进程
3 sleep 10000
4
5 # 2. 按下 Ctrl+Z 挂起进程（终端快捷键操作）
6
7 # 3. 将挂起的进程放到后台继续运行
8 bg
9
10 # 4. 查找目标进程（带完整命令行验证）
11 pgrep -af "sleep 10000"
12
13 # 5. 精准终止目标进程
14 pkill -f "sleep 10000"
```

运行结果：完成要求，`jobs` 检查显示无该进程

注：此题使用 kali 虚拟机，原因 `bash` 不自带 `pgrep` 和 `pkill` 命令，需安装 `procs`。



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sleep 10000  
^Z  
zsh: suspended sleep 10000  
(kali@kali)-[~]  
$ bg  
[1] + continued sleep 10000  
(kali@kali)-[~]  
$ pgrep -af sleep 10000  
2514 sleep 10000  
(kali@kali)-[~]  
$ pkill -f sleep 10000  
[1] + terminated sleep 10000  
(kali@kali)-[~]  
$ jobs  
(kali@kali)-[~]  
$
```

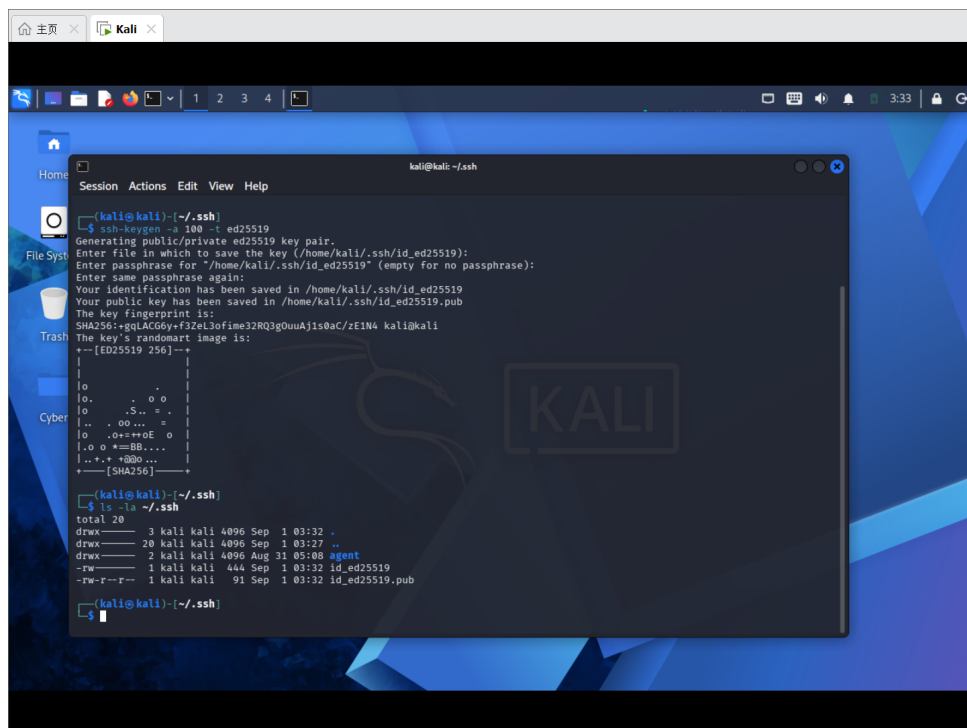
实例六运行结果

实例 7（命令行环境）

实验题目：(虚拟机) 进入 `./ssh/`，检查你是否已经有一对 SSH 密钥。如果没有，就用 `ssh-keygen -a 100 -t ed25519` 生成一对。建议给密钥设置密码，并配合 `ssh-agent` 使用。

```
1 # TODO: 实例7代码汇总
2 cd ~/.ssh
3 ls -la
4
5 #创建密钥
6 ssh-keygen -a 100 -t ed25519
7
8 #将私钥加入 ssh-agent
9 eval "$(ssh-agent -s)"
10 ssh-add ~/.ssh/id_ed25519
11 ssh-add -l
```

运行结果：创建密钥成功，ssh-agent 成功添加私钥



实例七运行结果

```

drwx----- 2 kali kali 4096 Aug 31 05:08 agent
-rw----- 1 kali kali 444 Sep 1 03:32 id_ed25519
-rw-r--r-- 1 kali kali 91 Sep 1 03:32 id_ed25519.pub

(kali@kali)-[~/ssh]
$ eval "$(ssh-agent -s)"
Agent pid 622909

(kali@kali)-[~/ssh]
$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/kali/.ssh/id_ed25519:
Identity added: /home/kali/.ssh/id_ed25519 (kali@kali)

(kali@kali)-[~/ssh]
$ ssh-add -l
256 SHA256:+gqLACG6y+f3Zel3ofime32RQ3g0uuAj1s0aC/zE1N4 kali@kali (ED25519)

(kali@kali)-[~/ssh]
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAyLL3Bh98t3hhRzp3Dk5GZAcB8fq0nNkJmP1Vgdda8g kali@kali

```

实例七运行结果

实例 8

实验题目：

.编辑 `ssh/config`，加入下面这样的配置项：

```

Host vm
  User username_goes_here
  HostName ip_goes_here
  IdentityFile ~/.ssh/id_ed25519
  LocalForward 9999 localhost:8888

```

1 # TODO：实例8无代码汇总

运行结果：如下图

```

(kali@kali)-[~/ssh]
$ gedit config
** (gedit:628366): WARNING **: 03:46:47.166: Could not load Peas repository: Typelib file for namespace
not found
** (gedit:628366): WARNING **: 03:46:47.166: Could not load PeasGtk repository: Typelib file for namesp
'1.0' not found

(kali@kali)-[~/ssh]
$ chmod 600 ~/.ssh/config

(kali@kali)-[~/ssh]
$ ls -l ~/.ssh/config
-rw----- 1 kali kali 123 Sep 1 03:49 /home/kali/.ssh/config

(kali@kali)-[~/ssh]
$ ssh -G vm | grep -E 'user |hostname |identityfile|localforward'
user kali
hostname 192.168.204.131
canonicalizehostname false
localforward 9999 [localhost]:8888
identityfile ~/.ssh/id_ed25519

```

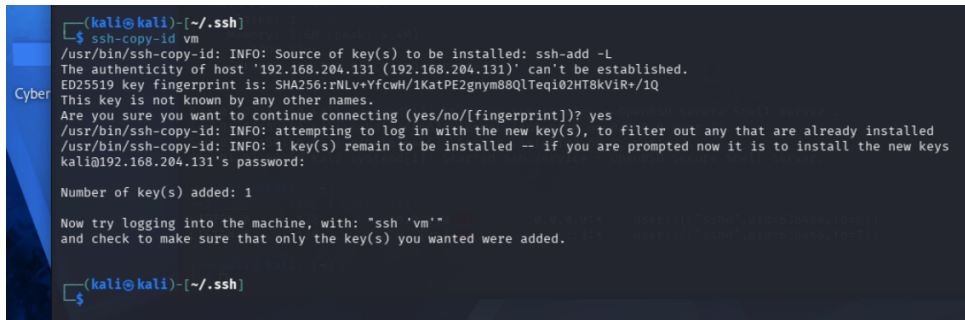
实例八运行结果

实例 9

实验题目：使用 `ssh-copy-id vm` 把你的 SSH 公钥复制到服务器上。

1 # TODO: 实例9无代码汇总

运行结果：复制成功



```
(kali@kali)~/.ssh$ ssh-copy-id vm
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ssh-add -L
The authenticity of host '192.168.204.131 (192.168.204.131)' can't be established.
ED25519 key fingerprint is: SHA256:rNLv+YfcwH/1KatPE2gnyM88QLTeqi02HT8kViR+/1Q
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kali@192.168.204.131's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'vm'"
and check to make sure that only the key(s) you wanted were added.

(kali@kali)~/.ssh$
```

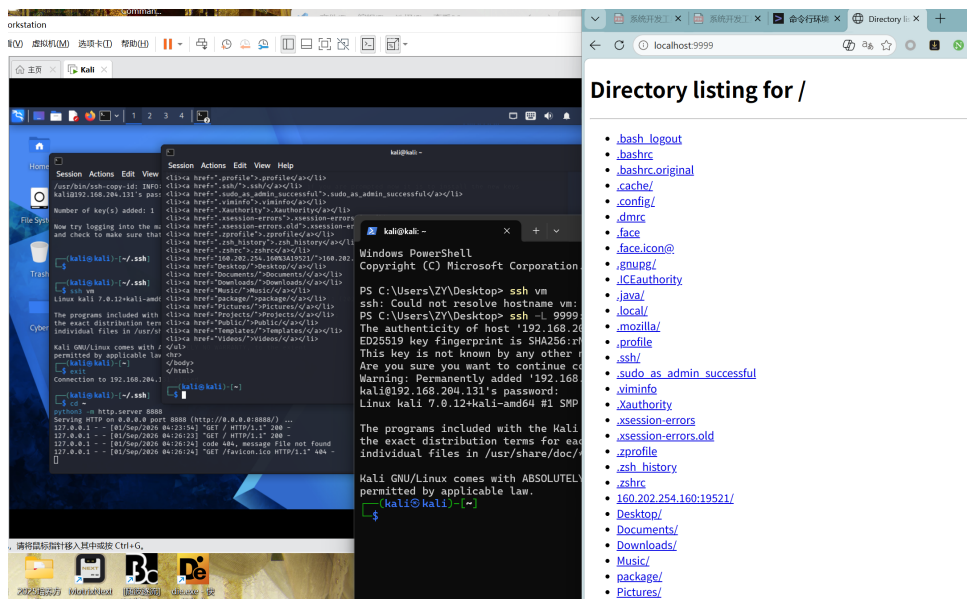
实例九运行结果

实例 10

实验题目：在虚拟机里执行 `python -m http.server 8888` 启动一个 Web 服务器。然后在你自己的机器上访问 `http://localhost:9999`，确认自己能访问虚拟机里的这个服务。

```
1 # TODO: 实例10代码汇总
2 #在kali中
3 python3 -m http.server 8888
4
5 #在本机中
6 ssh -L 9999:localhost:8888 kali@192.168.204.131
```

运行结果：本机成功访问 localhost:9999



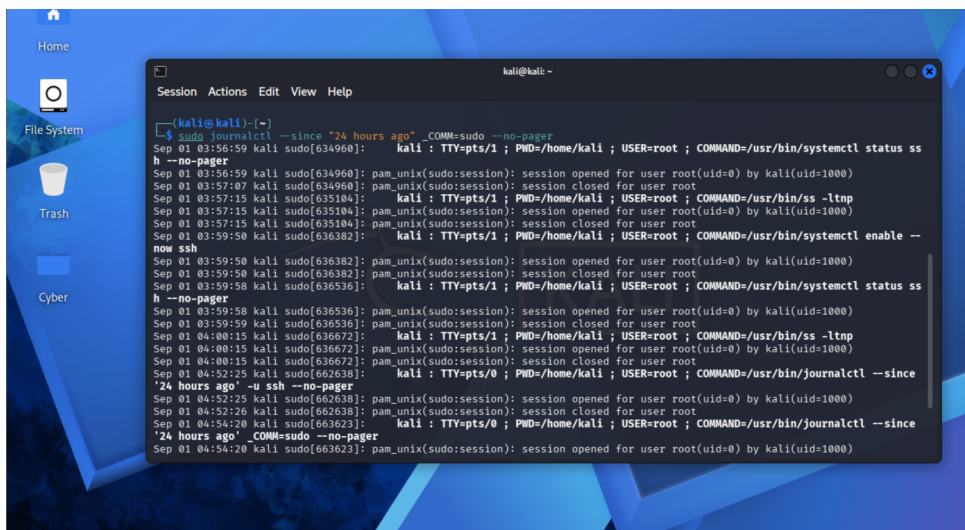
实例十运行结果

实例 11

实验题目：使用 Linux 上的 `journalctl` 或 macOS 上的 `log show` 命令来获取最近一天中超级用户的登录信息及其所执行的指令。如果找不到相关信息，您可以执行一些无害的命令，例如 `sudo ls` 然后再次查看。

```
1 # TODO: 实例 11 代码汇总
2 sudo journalctl --since "24 hours ago" _COMM=sudo --no-pager
```

运行结果：查看了通过 `sudo` 执行的命令。



实例十一运行结果

课后阶段提交记录

[图片待补充] `week\Week_commit_kehou.png`

课后阶段 `commit` 记录

3 解题感悟

本周实验涉及命令行环境、开发工具、程序调试、自动化测试、性能分析以及 SSH 远程操作等多个方面。

实验中遇到的问题主要包括后台进程状态变化、SSH 服务未启动、Windows 与 Kali 配置环境不同以及 SSH 主机别名无法解析等。通过查看错误信息、检查服务状态和分析命令输出，我逐渐形成了先定位问题、再提出解决方案的排错习惯。总体而言，本周实验将理论知识与实际操作结合起来，提高了我对 Linux 命令行、开发调试工具和系统服务的综合使用能力。